
KERNEL BORDERLANDS: AN eBPF-BASED BEHAVIORAL DETECTION AND MULTI-AGENT CONTAINMENT ARCHITECTURE FOR LINUX SERVER WORKLOADS

Pardhu Sri Rushi Varma Konduru
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040089@mallareddyuniversity.ac.in

Tejaswini Kandukoori
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040077@mallareddyuniversity.ac.in

Karthik Kanne
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040080@mallareddyuniversity.ac.in

Rupa Yeshvitha Karedla
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040082@mallareddyuniversity.ac.in

ABSTRACT

Kernel Borderlands (KB) is a kernel-level runtime defense framework for Linux composed of five implemented subsystems — an eBPF-based Ring-0 sensor (`kb-core`), a Go control plane for scoring, storage, and enforcement (`kb-control-plane`), a Python multi-agent swarm for adaptive response (`kb-aads`), a stateless Rust integrity watchdog (`kb-checker`), and a set of operator interfaces (`kb-op`) — plus a briefly-noted extended hardware feature developed in parallel by a dedicated sub-team in a separate repository, the Node Out-of-Band Module for Attestation & Defense (NOMAD), intended to eventually move the system’s last line of defense off the monitored host entirely. This paper documents the design, implementation history, and current status of every subsystem, including defects found and fixed along the way and defects still open, and gives a unified mathematical formulation of the framework’s learned and rule-based decision procedures: the Containment and Jury reinforcement-learning policies, Judge/Jury/Executor quorum consensus, the courthouse severity ladder for supervising the agent swarm itself, and the Shannon-entropy/CUSUM statistics underlying beaconing and slow-exfiltration detection. Relative to the project’s earlier state, this revision reports newly trained and live-verified components (a Jury consensus policy reaching 100.0% held-out test accuracy and 0% false-negative rate on 908 samples), newly implemented swarm roles (rule-based Patroller and Healer agents, a mechanically-sequenced Containment militia, dedicated signal-relay actors, and a `SwarmRegistry`-backed courthouse oversight mechanism for the agent swarm itself), a signed critical-workload policy format using Ed25519, and a severity-escalation and exfiltration-detection pipeline on the control plane. A source-level security audit conducted across four of the five implemented subsystems re-verified every previously reported open defect directly against current source rather than carrying old claims forward; the great majority — an unauthenticated HTTP API, a telemetry-reset containment bug, an LSM fail-open ordering issue, three JJE consensus defects, and more — turned out to already be fixed in team history that simply predated when this audit’s text was first drafted, and one genuinely open defect (an audit-chain integrity check silently masking a class of I/O error) is fixed in this revision with a dedicated regression test. Where a subsystem or feature is incomplete, stubbed, or under active parallel development elsewhere (NOMAD) at the time of writing, we say so explicitly rather than omitting it or implying a maturity the code or the design review does not support.

Keywords eBPF · Linux security · runtime containment · multi-agent systems · reinforcement learning · intrusion detection · distributed systems · hardware root of trust · trusted platform module

Code availability. The full source referenced throughout this paper is available at <https://github.com/pardhuvarmax/kernel-borderlands> [Kernel Borderlands Team, 2026].

1 Introduction

Runtime defense for Linux hosts increasingly relies on kernel-level observability: extended Berkeley Packet Filter (eBPF) programs attached to LSM hooks, tracepoints, and kprobes observe process behavior at Ring 0 with low overhead [Gregg, 2019], building on packet-filtering foundations dating back to the original BSD Packet Filter. Observability alone is not response — a runtime defense system must also decide, act, and be able to prove that its own decision trail has not been tampered with after the fact, and, if the goal is to defend against a full kernel compromise rather than merely detect one, it must eventually confront the fact that a Ring-0 attacker shares every software component’s trust domain, watchdogs included. Kernel Borderlands (KB) [Kernel Borderlands Team, 2026] is organized around an observe-score-decide-enforce-audit loop, implemented across five cooperating, independently substantial subsystems — kb-core (§5), kb-control-plane (§6), kb-aads (§7), kb-checker (§8), and kb-op (§10). A hardware-anchored external trust module addressing that same full-kernel-compromise point directly, NOMAD (§8.5), is under active parallel development by a dedicated hardware sub-team in its own separate repository and is noted only briefly here as an extended feature, not as one of this paper’s five central subsystems.

At the time of writing the repository holds 373 commits spanning approximately four months of active development. This paper’s aim is to document KB as it currently stands, subsystem by subsystem, including what each subsystem does, notable defects discovered and corrected during its development, and — honestly — what remains incomplete. §2 gives a system-level architecture overview with the full inter-process dataflow and socket topology. §3 summarizes the feature set as a single reference table. §4 collects, in one place, the mathematical formulation behind every learned or statistically-driven decision procedure in the system, so that later sections can refer back to it rather than re-deriving notation. §11 reports a source-level security audit conducted across four of the five implemented subsystems, updated in this revision to reflect fixes made since. §12 summarizes per-subsystem status in one table. We do not attempt to make any single subsystem the centerpiece of this report; each received comparable engineering effort and is treated accordingly.

2 System Architecture

Figure 1 shows the complete inter-subsystem dataflow. Sensor telemetry travels one direction only, from kb-core’s userspace bridge to kb-control-plane over `/run/kb/kbd.sock`; containment and sensitive-path/rule/CWP-workload pushes travel the opposite direction over a separate socket, `/run/kb/kbct.sock` (§5.3). Operator interfaces and the agent swarm reach the control plane’s gRPC surface over `/run/kb/kba.sock`; kb-checker exposes its own independent diagnostic gRPC service over `/run/kb/kbc.sock`. All four sockets are Unix domain sockets, root-owned, mode 0660 — there is no TCP listener anywhere in this dataflow, by design (§8, §8.5).

2.1 Socket Topology

Table 1 lists every Unix domain socket in the system, its owning process, and its direction. The `kbd.sock/kbct.sock` split (§5.3) exists specifically so a telemetry-volume burst can never stall or kill containment delivery — a real, previously-latent availability bug this split fixes, not a precautionary design choice made without cause.

Table 1: Unix domain sockets under `/run/kb/`.

Socket	Owner	Role
<code>kbd.sock</code>	<code>kbd</code>	Sensor → control plane telemetry. One direction only.
<code>kbct.sock</code>	<code>kbd</code>	Control plane → sensor: containment commands, sensitive-path/rules/CWP-workload pushes.
<code>kba.sock</code>	<code>kbd</code>	gRPC IPC: operator interfaces, kb-aads.
<code>kbc.sock</code>	<code>kb-checker</code>	Diagnostic gRPC for TUIs/CLIs, independent of <code>kbd</code> .

Figure 2 traces the same four sockets down to the actual pipeline stage each one connects, on both sides of the boundary — not just which process owns which socket, but what specifically happens to a byte of telemetry or a containment decision as it crosses it.

3 Feature Set

Table 2 summarizes the implemented feature set by subsystem and status, consolidating features discussed in detail in their respective sections. “Live-verified” means the feature was exercised against a running instance of the relevant component (not only a unit test in isolation) during this project’s development; “proposed” means design-only, with no code.

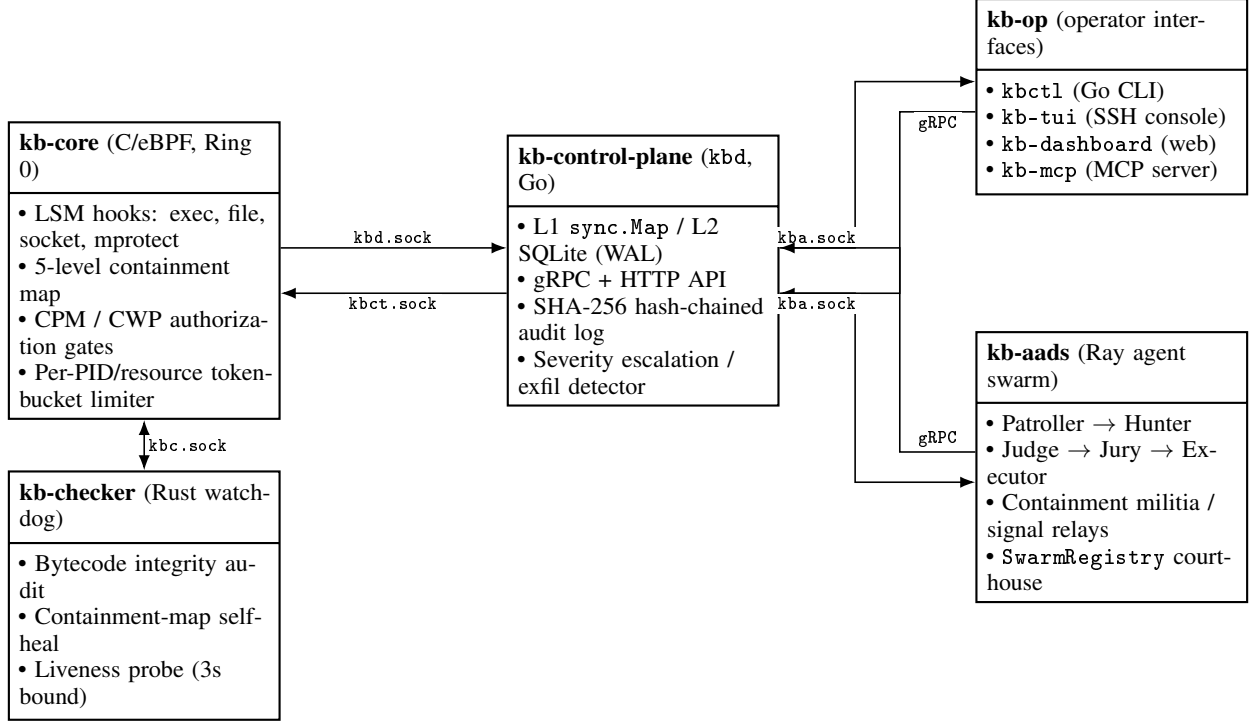


Figure 1: KB dataflow architecture — the five subsystems this paper covers in depth, with each box’s own internal working sketched below its name. Sockets are implemented and live-verified; each socket pair (kbd.sock/kbct.sock to kb-core, kba.sock to kb-op/kb-aads, kbc.sock to kb-checker) is bidirectional but drawn as two one-way arrows to make direction explicit (Table 1). NOMAD, an extended hardware feature developed in a separate repository (§8.5), is omitted here since it has no role in this dataflow.

4 Mathematical Formulation

This section collects every learned or statistically-driven decision rule in KB in one place. Each subsection restates, briefly, where the rule is used; the corresponding system section gives full context.

4.1 Containment Level Selection (Reinforcement Learning)

Containment’s decision (§7.3) is a single-step Markov Decision Process over a 5-dimensional observation $o = (\text{score}, \text{zone}, u_{\text{root}}, \Delta_{\text{score}}, e) \in \mathbb{R}^5$, a 5-way discrete action $a \in \{0, 1, 2, 3, 4\}$ matching `ContainmentLevel` (NONE through TERMINATE), and reward

$$r(a, t) = 1 - 0.3 |a - t| - 0.2 |a - t| \cdot \mathbb{I}[a < t] \quad (1)$$

against a labeled target level t , penalizing under-containment ($a < t$) more heavily than over-containment. The policy is a PPO actor [Schulman et al., 2017] trained via RLlib [Liang et al., 2018] on Ray [Moritz et al., 2018]; inference at serving time is $a^* = \arg \max_a \pi_\theta(a | o)$, taken from the policy’s action-distribution logits with no exploration noise.

4.2 Jury Vote Selection (Reinforcement Learning)

Jury’s decision (§7.4) reuses Containment’s exact observation space $o \in \mathbb{R}^5$ and dataset, coarsened to a binary target $t' = \mathbb{I}[t > 0]$ (any nonzero containment level $\Rightarrow$ CONTAIN). The action space is $a' \in \{\text{ALLOW}, \text{CONTAIN}\}$ and the reward, matching kb-aads’s pre-existing reward-signal table, is

$$r'(a', t') = \begin{cases} +1.0 & a' = \text{CONTAIN}, t' = \text{CONTAIN} (\text{TP}) \\ -0.5 & a' = \text{CONTAIN}, t' = \text{ALLOW} (\text{FP}) \\ +0.1 & a' = \text{ALLOW}, t' = \text{ALLOW} (\text{TN}) \\ -1.0 & a' = \text{ALLOW}, t' = \text{CONTAIN} (\text{FN}) \end{cases} \quad (2)$$

Where a full observation vector is unavailable, Jury falls back to a fixed-threshold rule on a bare confidence scalar $c \in [0, 1]$,

$$\text{vote}(c) = \begin{cases} \text{CONTAIN} & c > 0.75 \\ \text{ALLOW} & \text{otherwise} \end{cases} \quad (3)$$

which is the path every caller in the system uses today (§7.4).

4.3 JJE Quorum Consensus

Given a set of n weighted votes $\{(v_i, w_i)\}_{i=1}^n$ from a dynamically-spawned Jury pool, $v_i \in$

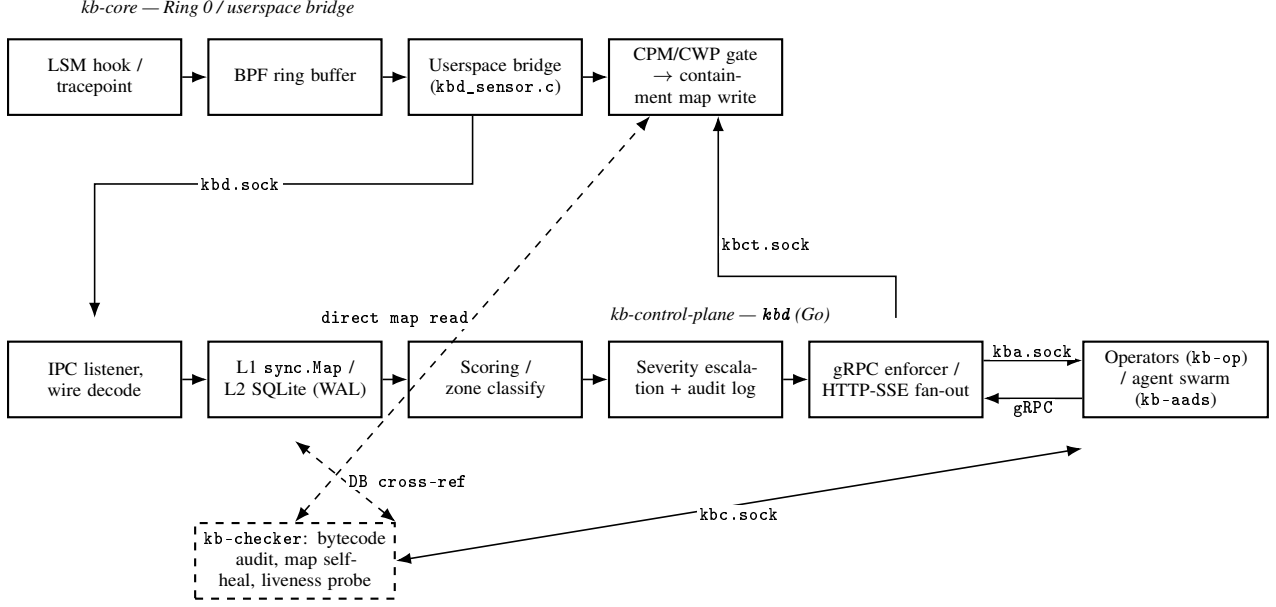


Figure 2: Socket topology traced to internal pipeline stages, not just socket ownership (contrast Table 1). Solid arrows within a lane are in-process control/data flow (function calls, eBPF ring-buffer submit/consume); solid arrows crossing a lane boundary are the labeled Unix domain socket. Dashed arrows are kb-checker’s independent, out-of-band verification paths (§8) — it reads kernel state and the L2 database directly to cross-check them, rather than trusting either side’s own report of itself.

Table 2: Feature set by subsystem, condensed. See referenced sections for detail.

Subsystem	Feature	Status
kb-core	5-level containment (§5)	Verified
	CPM / CWP gates (§5)	Verified
	Rate limiting (§5)	Verified
	socket split (§5.3)	Verified
	CO-RE/BTF portability	Done
kb-control-plane	L1/L2 storage (§6.1)	Verified
	Audit log (§6.2)	Verified
	Policy hot-reload (§6.3)	Verified
	Signed CWP policy (§6.4)	Verified
	Severity escalation (§6.5)	Verified
	Exfiltration detection (§6.6)	Verified
kb-aads	Containment RL (§7.3)	Verified
	Jury RL (§7.4)	Verified
	Patroller / Healer (§7.5)	Verified
	Militia (§7.6)	Verified
	Signal relays (§7.6)	Verified
	Registry + courthouse (§7.7)	Verified
	Hunter logic	Stub
kb-checker	Integrity/self-heal (§8)	Verified
	Liveness audit (§8)	Verified
	Boot gating (§8)	Verified
kb-op	kbctl CLI (§10)	Verified
	kb-tui console (§10)	Verified
	kb-dashboard (§10)	Verified
	kb-mcp server	Not audited

{CONTAIN, ALLOW}, quorum for containment is reached iff

$$\frac{\sum_{i:v_i=\text{CONTAIN}} w_i}{\sum_{i=1}^n w_i} > \frac{1}{2}, \quad (4)$$

defined as false on $n = 0$ (every juror in the round failed) rather than raising a division error. In the current implementation every $w_i = 1.0$ (unweighted majority); the formula is written in weighted form because the wire format already carries a per-vote weight field.

4.4 Beaconing Detection: Timing Entropy

For a sliding window of the most recent $\min(N, 100)$ inter-connection intervals $\{\Delta t_1, \dots, \Delta t_N\}$ for one (pid, destination address, destination port) flow, each interval is binned into one of five fixed buckets $B = \{(0, 1], (1, 5], (5, 10], (10, 30], (30, \infty)\}$ seconds, and Shannon entropy [Shannon, 1948] is computed over the resulting empirical bin distribution $p_j = n_j/N$:

$$H = - \sum_{j=1}^5 p_j \log_2 p_j. \quad (5)$$

$H \rightarrow 0$ indicates high timing regularity (a rigid periodic timer, characteristic of a beacon); H is undefined below ten samples and a sentinel value of 3.0 (“unprofiled, treat as high/benign”) is returned instead. The detector’s default beacon threshold is $H < 1.2$.

4.5 Slow-Exfiltration Detection: CUSUM over Connection Rate

For the same flow, connection events are aggregated into fixed 60-second buckets; let x_i be the connection count in bucket i . A running mean μ_i and variance are maintained online via Welford’s algorithm (no raw history retained):

$$\delta_i = x_i - \mu_{i-1}, \quad \mu_i = \mu_{i-1} + \frac{\delta_i}{i}, \quad (6)$$

$$M_{2,i} = M_{2,i-1} + \delta_i (x_i - \mu_i), \quad \sigma_i = \sqrt{\frac{M_{2,i}}{i-1}}. \quad (7)$$

A one-sided upper CUSUM statistic [Page, 1954] is then updated with slack $k = \sigma_i/2$:

$$S_i = \max(0, S_{i-1} + (x_i - \mu_i - k)), \quad (8)$$

and flagged once $S_i > 5\sigma_i$, meaningful only once at least five buckets have closed (an early verdict off a near-empty baseline would be noise, not signal). An EXFIL_BEACON_SUSPECTED alert fires only when *both* $H < 1.2$ and $S_i > 5\sigma_i$ hold simultaneously for the same flow, combining a fine-grained timing signal with a coarser volume-shift signal specifically to suppress the false positives either alone would produce, and is debounced to at most one alert per flow per five minutes. This implementation’s own stated fidelity limitation: x_i is a connection-frequency proxy, not true byte volume, since kb-core’s connect() telemetry carries no payload-size field (§6.6).

4.6 CWP Severity Escalation

A raw composite score $s \in [0, 100]$ (already computed upstream by the Behavior Engine, not re-derived here) is bucketed into one of four ordered tiers by

$$\text{tier}(s) = \begin{cases} \text{CRITICAL} & s \geq 90 \\ \text{HIGH} & 75 \leq s < 90 \\ \text{MEDIUM} & 50 \leq s < 75 \\ \text{LOW} & s < 50 \end{cases} \quad (9)$$

and escalated exactly one tier, capped at CRITICAL, when the alerting process’s executable basename matches a registered Critical Workload Protection (CWP) entry (§6.5). This basename match is an explicitly documented, lower-stakes trade-off for alert routing only — the actual containment-exemption decision uses kb-core’s kernel-side path-resolved match (§5), unaffected by this rule.

4.7 JJE Courthouse Severity Ladder

For a registered sub-agent, let e be its cumulative error-count, $\sigma \in \{\text{active}, \neg\text{active}\}$ its reported status, and u_0, u_1 its uptime read before and after a liveness window of length τ (default 3s). The courthouse classifier (§7.7) is

$$\text{severity} = \begin{cases} \text{terminate} & \text{agent unregistered} \\ \text{restart} & \sigma \neq \text{active} \vee e \geq \theta_e \vee u_1 \leq u_0 \\ \text{healthy} & \text{otherwise} \end{cases} \quad (10)$$

with default error-count threshold $\theta_e = 5$. The classifier never automatically returns revoke or (for a still-registered agent) terminate — those tiers require a peer-baseline output-drift signal and an authorization-boundary-violation signal respectively, neither of which this codebase currently computes (§7.7); Equation (10) is stated exactly as implemented, not as the ladder’s eventual full form.

4.8 Audit Log Integrity

Each audit row i commits to every prior row via a SHA-256 hash chain,

$$h_i = \text{SHA256}(f_i \parallel h_{i-1}), \quad h_0 = \mathbf{0}, \quad (11)$$

where f_i is the row’s serialized field tuple. Mutating or deleting any row $j < i$ changes h_j and therefore every h_k for $k \geq j$ under the second-preimage resistance of SHA-256, making a chain break detectable by recomputing Equation (11) end to end (§6.2).

5 kb-core: eBPF Sensor and Enforcement Primitives

kb-core is a C/eBPF subsystem, built with CO-RE (Compile Once, Run Everywhere) via BTF for cross-kernel portability without per-kernel-version recompilation. It is the only subsystem with a presence in Ring 0. A full make of kb-core, including eBPF bytecode compilation for every collector (kb_process, kb_syscall, kb_privilege, kb_file, kb_network, kb_memory) and the combined kbd_sensor binary, was verified clean at the time of writing.

5.1 Hook Points and the Containment Model

kb-core attaches LSM hooks and tracepoints covering process execution (bprm_check_security), file access (file_open), network connect/bind, memory-protection changes (file_mprotect), and credential changes (commit_creds), and maintains a five-level containment state per monitored process, uniform across all hooks, escalating through the standard Linux primitives of cgroups, seccomp, and namespaces [Kerrisk, 2010]: NONE (no entry; every hook passes through), CGROUP (resource throttling only, no LSM blocking), SECCOMP (blocks new network connections/binds and process exec), NAMESPACE (adds RWX/exec memory-protection blocking on top of SECCOMP), and TERMINATE (blocks everything interceptable, combined with a userspace SIGKILL as a belt-and-suspenders measure). Every hook returns a uniform -EPERM rather than hook-specific error codes, so the kernel presents a consistent “Operation not permitted” regardless of which containment layer intervened.

5.2 CPM and CWP: Authorization Gates on Containment Writes

Two authorization layers sit in front of containment map writes, both implemented in userspace C (kbd_

sensor.c's `cpm_classify()/cwp_classify()` rather than in-kernel eBPF — a deliberate, documented deviation from an earlier design sketch, since the real last-gate-before-enforcement choke point in this codebase is `handle_incoming_containment_cmd()`, itself userspace. **CPM (Critical Process Module)** (Algorithm 1) refuses to contain PID 1, kernel threads (detected by a failed `/proc/<pid>/exe` readlink), any PID in `protected_pids_map`, or any process whose live-resolved executable path matches `protected_exec_paths_map` — the last check exists specifically as a race-safety net against a just-executed protected binary not yet reflected in the PID map. Its compiled-in floor (`populate_protected_exec_paths()`) now covers `systemd`, `dbus-daemon`, `NetworkManager`, `kb-checker`, `kbd`, `kbagents`, `kbopd`, and `kbopt` — closing a prior gap where KB's own watchdog or policy engine could in principle be recommended for containment through KB's own pipeline. Both backing maps are marked `BPF_F_RDONLY_PROG` so no in-kernel BPF program can write either, only userspace, and `protected_pids_map` entries are populated at exec time and cleared at exit time so a reused PID cannot inherit a predecessor's protection.

CWP (Critical Workload Protection) (Algorithm 2) is evaluated strictly after CPM returns allow, never merged with it. Each registered workload carries a single `identity_tier` byte, not two independent lookup mechanisms: `PATH` tier rejects on a bare path/PID match, while `HASH` tier defers to a SHA-256 content check computed at decision time (hashing cannot happen in-kernel; registration only stores the tier flag and expected digest). A hash *mismatch* on a `HASH`-tier entry does not itself block anything — it logs a distinct “Identity Verification Failed” security event and falls through to ordinary containment eligibility, since the process is thereby confirmed *not* to be the protected workload it claimed to be. `protected_workloads_map` is sized to 8192 entries with no compiled-in floor, since protected workloads are organization-specific by design. Both gates were verified against the live running sensor, not merely built: a live test confirmed that a containment command for a protected PID was rejected correctly while the connection was simultaneously saturated with unrelated telemetry.

Algorithm 1: CPM classification (`cpm_classify`, `kbd_sensor.c:677-702`)

```

1: if readlink(/proc/PID/exe) fails then
2:   return REJECT_KERNEL_THREAD
3: if PID = 1 then return REJECT_PID1
4: if PID ∈ protected_pids_map then
5:   return REJECT_PROTECTED_PID
6: path ← readlink(/proc/PID/exe)
7: if path ∈ protected_exec_paths_map then
8:   return REJECT_PROTECTED_EXEC
9: return ALLOW    (only reached if steps
1-8 all miss)
```

Algorithm 2: CWP classification (`cwp_classify`, `kbd_sensor.c:964-1045`)

```

1: tier ← lookup(protected_workloads_map, PID)
2: if tier not found then
3:   path ← readlink(/proc/PID/exe); if fails, return ALLOW
4:   tier ← lookup(protected_workload_paths_map, path)
5:   if tier not found then return ALLOW
6:   if tier = PATH then return REJECT_PROTECTED_PATH
7:   {tier = HASH}: recompute path if not already resolved;
8:   if resolution fails then return REJECT_PROTECTED_HASH (fail-closed)
9:   entry ← registry lookup(path); if none, return REJECT_PROTECTED_HASH
10:  digest ← SHA256(file contents at path)
11:  if digest ≠ entry.expected_hash then
12:    log(“Identity Verification Failed”); return ALLOW
13:  else return REJECT_PROTECTED_HASH
```

5.3 The `kbd.sock` / `kbct.sock` Split

Telemetry and containment commands were originally multiplexed on a single, hand-rolled socket connection between the sensor and the control plane. This proved to be a latent availability bug: a telemetry-volume burst could fill the connection's send buffer, `write()` would return `EAGAIN`, and the sensor's error handling misread this as a dead connection and closed it — taking containment delivery down as collateral damage of an unrelated telemetry spike. The fix splits control traffic (containment commands, sensitive-path, rules, and CWP-workload pushes) onto a dedicated `kbct.sock`, leaving `kbd.sock` telemetry-only, and additionally ignores `SIGPIPE` process-wide so a closed peer cannot kill the sensor via signal regardless of the split.

5.4 Rate Limiting

Telemetry rate limiting (Algorithm 3) uses a hybrid token bucket per `{tgid, start_time, resource_id}` key in a 10240-entry BPF LRU hash map — keying on the task's creation timestamp alongside its `tgid`, not `tid` and resource alone, specifically to prevent a PID-reuse collision from inheriting a stale or already-exhausted predecessor's bucket. Resource IDs are call-site specific: the syscall number for generic syscall telemetry, a packed `(daddr<16) | dport` for network connect/bind, and the relevant flag word for `mmap/mprotect` — specifically to prevent an evasion technique in which an attacker distributes activity across many distinct resources to stay under a single flat per-PID limit. Bucket capacity is 100 tokens, refilling at one token per 10ms (100 tokens/second); a bucket

that recovers tokens after having dropped events emits one `KB_EVT_DROPPED_ TELEMETRY` summary event carrying the suppressed count, rather than silently discarding the fact that throttling occurred. Tier-1 events — privilege-escalation (`commit_creds`), sensitive-capability probes, and TLS-plaintext capture — are exempted from throttling structurally: their handlers simply never call the throttle function before submitting to the ring buffer, not via a bypass flag. (LSM enforcement hooks, §5, have no corresponding telemetry event at all on denial — they return a kernel error code directly and never touch the ring buffer or the rate limiter, so “exempting” a denial from throttling is not applicable to them.)

Algorithm 3: Token-bucket throttle (`kb_rate_limit_throttle`, `kbd_sensor.bpf.c:235-287`)

```

1: key ← {tgid, task.start_time,
  resource_id}
2: bucket ← lookup(lru_map, key)
3: if bucket = NULL then
4:   bucket ← {tokens: 100, last: now,
  dropped: 0}; insert
5: elapsed ← now − bucket.last
6: if elapsed > 10ms then
7:   bucket.tokens
  ← min(100, bucket.tokens + ⌊elapsed/10ms⌋)
8:   bucket.last ← now
9: if bucket.dropped > 0 and bucket.tokens
  > 0 then
10:  emit DROPPED_TELEMETRY(count =
  bucket.dropped); bucket.dropped ← 0
11: if bucket.tokens > 0 then bucket.tokens
  −= 1; return ALLOW
12: else bucket.dropped += 1; return
  THROTTLE

```

5.5 New Wire Message: Network-Flow Timestamps

A new userspace-only wire message, `KB_WIRE_MSG_ NET_FLOW` (message type 9, a 22-byte {pid, daddr, dport, ts_ns} frame), is sent from `kbd_sensor`’s existing `KB_EVT_NETWORK_CONNECT` handler to feed §6.6’s exfiltration detector. This required no eBPF or kernel-side change — the timestamp is already available in the existing userspace event path, per the exfiltration-detection design’s own recommendation to avoid adding Ring-0 overhead for a control-plane statistical computation.

5.6 Known Defects

A historical, critical defect is documented here for completeness: an early version of the sensitive-path LSM block was unconditional (applied to every process, not only contained ones), and the first time it fired correctly in the project’s history it blocked `/etc/sudoers/etc/shadow` reads system-wide, breaking `sudo`, `su`, `pkexec`, and login authentication simultaneously on the test host, recoverable only via a full external VM restart. This was fixed

by making the block containment-gated, consistent with every other hook. A related bug — the block technically never enforced anything even when intended to, because `bpf_d_path()` does not zero the tail of its output buffer past the NUL terminator, causing exact-match lookups against a fixed-size key to silently fail — was found and fixed by explicitly zero-filling the buffer. An earlier revision of this paper’s audit reported two further items as open that re-verification against current source shows are already resolved: `kb_lsm_file_open` checks the level-3+ block *before* resolving the path via `bpf_d_path()` (Algorithm 1’s sibling hook, same file), so a path-resolution failure cannot bypass full quarantine; and of the two genuinely PID-keyed telemetry maps (`kb_syscall_totals`, `kb_cred_prev`), both are explicitly deleted in `kb_handle_exit`. A third map, `kb_syscall_counts`, is composite-keyed (`pid<<32|syscall_nr`) rather than PID-keyed, so a single-PID delete on exit cannot target its entries without iterating the whole map; it uses `BPF_MAP_TYPE_LRU_HASH` instead, which ages out old entries under normal churn — a different, and adequate, mitigation for the same exhaustion concern, not an oversight.

6 kb-control-plane: Scoring, Storage, and Enforcement

`kb-control-plane` is a Go daemon (`kbd`) responsible for ingesting sensor telemetry, maintaining process state, exposing a gRPC and HTTP API, and issuing containment commands back to the sensor. Its Go test suite — `internal/audit`, `controlplane`, `detection`, `enforcement`, `ipc`, `policy`, `store` — comprises 128 passing tests with zero failures at the time of writing.

6.1 Storage: L1/L2 Hybrid

State is kept in a two-layer store, per this project’s ADR-1: an in-memory `sync.Map` (L1), authoritative for all reads, and an asynchronously-written SQLite database in WAL mode (L2), durable and single-writer (`SetMaxOpenConns(1)`) via a buffered channel, so that L1 writers never block on disk I/O even under load. On startup, L1 is restored from L2’s durable copy. PID-reuse safety is handled explicitly: a process-exit wire message flushes both the stale L1 entry and the L2 row immediately, so a later, unrelated process reusing the same PID cannot inherit a predecessor’s cached state.

6.2 The Audit Log

Every enforcement action, policy reload, and agent decision is recorded in a SHA-256 hash-chained append-only log (Equation (11), Algorithm 4): each row’s `entry_hash` commits to that row’s own fields joined with the *previous* row’s `entry_hash` (`prev_hash`) — the chain runs through `prev_hash`, not through a row hashing its own `entry_hash` recursively — rooted at the literal genesis value “genesis”. A `VerifyChain()` routine existed

correctly for some time before being wired to anything callable outside its own test — meaning the security property existed in code but delivered no operational value until an HTTP route (GET /api/audit/verify) and a gRPC VerifyAuditChain method were added to actually invoke it, alongside a `kbctl audit verify` command (§10). This audit’s own review of `VerifyChain()` found a real defect in exactly the direction that matters most for a security property: after its row-iteration loop exits, it never calls `Go’s rows.Err(). sql.Rows.Next()` returns false both on ordinary exhaustion *and* on an underlying driver/I/O error mid-cursor, with the real error stashed on `rows.Err()` rather than returned by `Next()` itself — so a genuine I/O error partway through verification is silently indistinguishable from a clean, complete scan, and `VerifyChain()` reports the chain valid (true) with a row count lower than the true total, rather than surfacing the error. This is a false-confidence bug, not a false-alarm one: it can under-report tampering exposure by claiming a shorter, unverified prefix is the whole story, not incorrectly flag an intact chain as broken. Fixed in this revision by checking `rows.Err()` immediately after the loop and returning it as a genuine failure rather than falling through to the success path (Algorithm 4), with a dedicated regression test using a fault-injecting database/sql/driver wrapper that fails the cursor mid-scan — a scenario the existing test suite’s row-scan-error test (itself a distinct, already-correctly-handled case) did not cover.

Algorithm 4: Audit chain log/verify
(internal/audit/audit.go)

```
Log(ts, action, subject, actor, reason):
1: content ← ts || action || subject || actor
  || reason || prev_hash
2: entry_hash ← SHA256(content)
3: append row; prev_hash ← entry_hash
VerifyChain():
4: rows ← SELECT * FROM audit_log ORDER BY
  id ASC
5: if query error then return (false, 0,
  err)
6: prev ← "genesis"; count ← 0
7: while rows.Next() do
8:   if scan error then return (false,
  count, err)
9:   expected ← SHA256(row fields || prev)
10:  if expected ≠ row.entry_hash then
11:    return (false, count, "chain
  broken")
12:  prev ← row.entry_hash; count += 1
13: if rows.Err() ≠ nil then return (false,
  count, err) {fix}
14: return (true, count, nil)
```

6.3 Policy Engine and Hot-Reload

`policy.yaml` configures per-process auto-terminate overrides and the sensitive-paths list consulted by `kb-core`.

Changing this configuration originally required a full daemon restart, which also dropped every sensor connection and forced a cold-start L1 rebuild. This was corrected by adding a `ReloadPolicy` gRPC method that re-reads `policy.yaml` and atomically swaps a freshly built `policy.Engine` into place under a `sync.RWMutex` — a full lock on the (rare) write path, a read lock on the (frequent, per-zone-transition) read path — verified during this audit to be genuinely race-free. A separate, previously-diagnosed-backwards misconfiguration is worth recording precisely because the first diagnosis was wrong: `kb-control-plane/config/policy.yaml` (the file `kbd`’s default `-policy` flag resolves to when run from its own documented working directory) carried the old, pre-internal/policy schema and silently produced empty, unenforced policies with no error — while the root-level `config/policy.yaml` was, and had always been, correctly schema-conformant. This was confirmed empirically (not by inspection) by running `policy.New` against both files directly, and fixed by replacing the daemon-default file’s content to match.

6.4 Signed Critical-Workload Policy

Critical Workload Protection’s local policy file, `workloads.yaml`, can now be signed: `SignWorkloadsFile/VerifyWorkloadsSignature` implement Ed25519 signing over a deterministic JSON serialization of the file’s `critical_workloads` and `policy_metadata` content, excluding the signature field itself. Passing `kbd -workloads-pubkey <path>` makes verification mandatory: a missing, invalid, or tampered signature is rejected outright and the previous known-good registry is retained rather than partially trusted. `kbctl workload keygen/sign` give operators an actual signing workflow; because `kbctl` is a separate Go module from `kb-control-plane`, its signing logic is a small, intentionally self-contained mirror of the control plane’s own implementation — Go’s internal-package visibility rule blocks a cross-module import of `kb-control-plane/internal/ipc` regardless of the module’s own `replace` directive, confirmed by testing the import directly rather than assumed. What this closes is local, host-resident policy signing; a networked central policy store (a separate, documented specification section) is explicitly not implemented, since no fleet-management server exists anywhere in this codebase for any subsystem.

6.5 CWP Severity Escalation

Alerts arising from a BORDERLANDS-zone transition are now bucketed into one of four severity tiers by raw score (Equation (9)) rather than the previous behavior of always reporting “CRITICAL” regardless of actual severity, and escalated one tier when the alerting process’s executable basename matches a registered CWP workload. The Go-side match is explicitly, and only, a basename match against `comm` (a 16-byte wire field) — the canonical, path-resolved, spoofing-resistant

match required for the actual containment-exemption decision happens kernel-side in `cwp_classify()` and is unaffected; this Go-side rule governs only alert severity and routing metadata (`protected_workload/owner_team/justification/ policy_id`, four new `Alert` proto fields).

6.6 Exfiltration and Beacons Detection

Implements the timing-entropy and CUSUM statistics of Sections 4.4 and 4.5 against the new `KB_WIRE_MSG_NET_FLOW` stream. An `EXFIL_BEACON_SUSPECTED` alert fires only when both signals agree, debounced to one alert per (pid, destination address, destination port) flow per five minutes — without this debounce, a sustained real beacon would re-alert and re-write an audit-log entry on every single subsequent connection indefinitely, a bug caught by a dedicated regression test before this shipped. Automatic `NAMESPACE`-level containment on a confirmed beacon is available but opt-in per process name via a new policy key, `auto_namespace_on_exfil_beacon` (default false), reflecting the same conservative default already applied to `auto_terminate` — a statistical heuristic carries real false-positive risk and should not silently gain enforcement authority by default.

6.7 Known Defects

A historical, critical defect: the wire-level constant for containment commands (`MsgTypeContainmentCmd`) was 3 on the Go side but the C sensor checked for 5, so every containment command issued via any operator interface was silently dropped, while the control plane’s own audit log recorded a successful entry regardless (the sensor never NACKs) — completely masking the failure until a live test against the running sensor exposed it. An earlier revision of this paper’s audit additionally reported five items as open that, on re-verification directly against current source for this revision, are already fixed in prior team history and were simply never re-checked before being written up: the HTTP isolate/restore API now requires a bearer token (`KB_HTTP_API_TOKEN`, constant-time compared, fails closed if unconfigured) and CORS is an explicit origin allowlist, not a wildcard; `UpsertProcessState` explicitly preserves an active PID’s containment level across routine telemetry updates rather than overwriting it; `SetContainment/SubmitAgentDecision` both have existence checks and per-agent call throttling, covered by dedicated tests; and the dashboard resolves its API host from `window.location.hostname` (overridable via `VITE_KB_API_BASE`), not a hardcoded `localhost:8080`. The audit-chain verifier’s `rows.Err()` gap (§6.2) was confirmed genuinely open and is fixed in this revision (Algorithm 4), with a dedicated fault-injecting-driver regression test.

7 kb-aads: The Agent Defense Swarm

kb-aads is a Python subsystem built on Ray, organized as a set of role-specialized actors intended to jointly monitor, investigate, and respond to threats. Its test suite comprises 68 passing tests across eight agent-role modules at the time of writing.

7.1 Actor Architecture

Roles are implemented as Ray actors subclassing a common `BaseAgent`: **Patroller** (baseline monitoring), **Hunter** (threat investigation), **Healer** (recovery decisions), **Containment** (enforcement-level selection), **Containment militia** (mechanical escalation execution, §7.6), **signal relays** (message dispatch, §7.6), and a **Judge/Jury/Executor (JJE)** consensus mechanism gating high-stakes decisions behind a quorum vote (Equation (4)) before **Executor** carries them out over gRPC to kb-control-plane. The actor scaffolding itself required a non-trivial fix early in its development: `BaseAgent` was originally decorated `@ray.remote` directly, which raises `ActorClassInheritanceException` the instant any concrete role subclass also applies `@ray.remote`. The fix removes the decorator from the base class and applies it individually per concrete subclass, with a `RemoteBaseAgent` convenience wrapper for roles with no dedicated subclass.

7.2 JJE Consensus

Judge opens a round when a Patroller-raised alert crosses a threshold, a dynamically-spawned Jury pool casts weighted votes (Equation (4)), and Executor carries out the resulting decision only after quorum. Two defects previously reported against this mechanism are, at the time of writing, confirmed fixed in the current source rather than open: Jury’s vote threshold now correctly operates on the same $[0, 1]$ confidence scale used everywhere else in the system (Equation (3), not the previously-reported unreachable 75.0 threshold against a 0–1 input), and `coordinate_consensus`’s Jury pool size is read from `config/agents.yaml` (`jury.pool_size`) rather than hardcoded, confirmed by a dedicated test asserting the exact spawn count for a configured value. A crashed juror’s exception is caught via `asyncio.gather(..., return_exceptions=True)`, so one failing actor degrades the round’s quorum computation rather than aborting it entirely — also confirmed by a dedicated test simulating two crashing jurors out of three.

Algorithm 5: JJE consensus round (`JudgeAgent.coordinate_consensus`)

```
1: pool ← spawn  $n = \text{jury\_pool\_size}$  fresh
   Jury actors
```

```

2: futures ←
[jury.evaluate_and_vote(alert) for jury in
pool]
3: results ← gather(futures,
return_exceptions = True)
4: votes ← [r for r in results if r is not
Exception]
5: if votes is empty then return (round
skipped, no quorum)
6: if quorum_reached(votes) {Equation (4)}
then
7:   executor.execute_quarantine(alert)

```

7.3 Containment: A Trained Reinforcement-Learning Policy

Of kb-aads’s decision-making roles, Containment was the first backed by a trained policy rather than a stub or an unreachable rule. Applying reinforcement learning to intrusion response specifically (as opposed to detection alone, the more heavily studied problem [Liao et al., 2013]) is an active area; we contribute a single, concrete, reproducible instance of it rather than a general treatment, formulated in Section 4.1.

Data. No real eBPF-captured attack corpus existed for KB at the time of this work, so real public data matching KB’s actual observation surface — Linux syscall sequences, not network flow, which an initially-tried NSL-KDD dataset was rejected for mismatching — was used instead. We mapped five of ADFA-LD’s six attack categories [Creech and Hu, 2013] onto five of KB’s own named attack scenarios (Table 3); KB’s memory_exploit scenario has no ADFA-LD equivalent and is not covered. The benign class combines ADFA-LD’s Normal traces with real /proc-derived telemetry sampled from a live development host.

Table 3: ADFA-LD category to KB attack-scenario mapping.

ADFA-LD category	KB scenario
Adduser	privilege_escalation
Hydra_FTP	credential_access
Hydra_SSH	lateral_movement
Java_Meterpreter	process_injection
Meterpreter, Web_Shell	reverse_shell

Training and a mid-training defect. An initial PPO run reached 97.8% overall test accuracy but only 60–73% on the CGROUP/SECCOMP classes (Table 4), root-caused to credential_access and lateral_movement sharing the same engineered event_type code and zone floor. Assigning each category a distinct code and retraining improved every attack category to 97–100% and reduced the under-containment rate to 0.0%, at the cost of NONE accuracy falling from 100% to 82.2% (Table 5) — a trade

we consider correct for a containment system, since a false positive on benign traffic costs a review action while a false negative on a real threat costs a missed compromise.

Table 4: Per-level test accuracy before and after the event_type fix.

Level	Before fix	After fix
NONE	100.0%	82.2%
CGROUP	60.0%	100.0%
SECCOMP	70.4%	100.0%
NAMESPACE	100.0%	97.1%
TERMINATE	93.3%	100.0%

Table 5: Overall test-set results before and after the fix.

	Before fix	After fix
Overall accuracy	97.8%	84.4%
Under-containment rate	1.1%	0.0%

Live verification. We verified the trained checkpoint three ways: offline test-set evaluation (above); a direct unit-level check against a live ContainmentAgent Ray actor for one synthetic observation per attack scenario; and an end-to-end run streaming real telemetry from KB’s actual eBPF sensor and control plane on a live host. The end-to-end check surfaced an integration defect distinct from the model: the control plane returns a zero-value default ProcessState (not an error) for an unscored PID, and a naive feature mapping misread the resulting default uid=0 as “running as root,” producing spurious high-severity decisions for untracked, usually short-lived processes — corrected by treating an empty comm field as the true not-found signal.

7.4 Jury: A Second Trained Policy

Jury’s binary CONTAIN/ALLOW decision (Section 4.2) is a strict coarsening of Containment’s own labeled question, so no new data collection was needed: the identical train.csv/val.csv/test.csv splits were reused with the target collapsed to $t' = \mathbb{I}[t > 0]$, and training used the same PPO/RLlib/Ray setup as Section 7.3. Table 6 reports held-out test performance from the checkpoint currently on disk, re-run for this revision rather than repeating an earlier session’s unlogged figures — training uses no fixed random seed, so exact percentages vary slightly run to run (an earlier run scored 99.8%/99.4% val/test on the same splits); the qualitative result (near-perfect accuracy, zero false negatives) has held across every run observed so far.

JuryAgent.evaluate_and_vote loads the resulting checkpoint the same inference-only way ContainmentAgent does (RLModule.from_checkpoint, no training stack in the agent process) and runs the trained policy only when a caller supplies a full observation vector; every caller and test in the

Table 6: Jury policy held-out test results (908 samples), from the checkpoint currently on disk.

Metric	Value
Validation accuracy	100.0%
Test accuracy	100.0%
CONTAIN recall	100.0%
False-negative rate	0.0%

system today supplies only a bare confidence scalar, so Equation (3)’s fixed-threshold rule remains the path actually exercised in practice until a caller starts passing full observations — stated plainly rather than implied to be already in production use.

7.5 Rule-Based Patroller and Healer

Patroller previously set a static status string on every tick and took no action; it now tracks a per-PID score and dispatches to a round-robin Hunter pool (injected at construction, mirroring how Judge receives its Executor reference) the moment a PID first crosses a configurable suspicious-score threshold (default 40.0), re-arming once the score drops back below it. Healer’s decision (§7.2’s reward-table shape — “was un-containing this PID actually safe?”) is *not* reinforcement-learned, and this is a stated, real gap rather than an oversight: that question needs restore-outcome ground truth (was the process actually safe N minutes after restoration?), which does not exist anywhere in this codebase — Containment/Jury’s training data has attack-category labels only, and the analyst-feedback loop that would eventually produce restore-outcome labels remains undesigned. Healer instead implements a genuine, conservative rule: restore only if zone is SAFE *and* score is below a safe threshold (20.0) *and* a minimum containment duration (60 ticks) has elapsed, with any missing or ambiguous signal defaulting to the least permissive reading (never accidentally RESTORE).

7.6 Containment Militia and Signal Relays

Two swarm roles previously existed only as design proposals, not code. **Containment militia** implements the design’s “squad” framing literally as an escalation sequence: a `MilitiaSquadLeadAgent`, given a target containment level $\ell \in \{1, \dots, 4\}$ for one PID, spawns one mechanical `MilitiaSquadMemberAgent` per stage (CGROUP→SECCOMP→NAMESPACE→TERMINATE, matching Containment’s own level ordering) dynamically per incident — mirroring Judge’s own per-round Jury-pool spawn pattern — and executes them strictly in sequence via `await`, so two stages can never race enforcement actions against the same PID. Each member is genuinely mechanical: it calls `ControlPlaneClient.set_containment` for exactly the one stage it owns, a real gRPC method that existed but had no live caller prior to this work. Escalation halts on the first stage failure, with no retry policy defined — a stated, real gap rather than a silent swallow. **Signal relays**

implement the design’s “dedicated messenger agent” framing as a long-lived, tick-driven actor round-robinning a message to a named destination pool, mirroring Patroller’s own pool-dispatch shape; one concrete route (to the Hunter pool) is wired at swarm start, while other agent pairs continue to call each other directly, an explicitly incomplete integration rather than a claim that every message now flows through a relay.

Algorithm 6: Militia escalation (`MilitiaSquadLeadAgent.command_squad`)

```

1: stages ← [CGROUP, SECCOMP, NAMESPACE,
TERMINATE][0:ℓ]
2: for stage in stages do
3:   member ← spawn
  MilitiaSquadMemberAgent(stage)
4:   result ← await member.execute(pid,
reason)
5:   {blocking await: no two stages ever
race on this pid}
6:   if not result.success then
7:     break (halt; no retry policy
defined)
8: return stage_reports
    
```

7.7 SwarmRegistry and JJE Courthouse Oversight

The swarm’s design calls for JJE to hold real authority to stop a rogue sub-agent via a graduated restart/revoke/terminate response (Equation (10)), which required an actor-lookup mechanism that did not previously exist: `RaySwarmOrchestrator`’s agent-handle dictionary lives in the driver process, unreachable by a remote `JudgeAgent`. A `SwarmRegistry` actor closes this gap; registration happens at each spawn site (the orchestrator, Judge’s per-round Jury pool, the militia squad lead’s per-stage members) rather than via each agent self-registering inside its own `start()`, giving identical coverage of dynamically-spawned actors with no change to `BaseAgent` or any existing agent subclass’s constructor. `JudgeAgent.assess_severity` and `enforce_verdict` implement Equation (10) and the corresponding mechanical response (stop-then-restart on the same actor; stop plus mark-revoked in the registry; `ray.kill` plus unregister), using two real, newly-tracked signals — a per-agent `error_count` and a genuine two-read liveness check. As Section 4.7 states plainly, the classifier never automatically reaches `revoke` or `terminate` for a still-live agent, since this codebase computes no peer-baseline drift or authorization-boundary-violation signal; the quorum-gating step the design calls for before acting on restart/revoke is likewise not implemented, since forcing `JuryAgent`’s existing threat-vote evaluator to answer a structurally different question (“is this sub-agent rogue?”) via a fabricated confidence input would not constitute a real vote.

Algorithm 7: Courthouse verdict
 (JudgeAgent.assess_severity + enforce_verdict)

```

1: handle ← registry.get_handle(agent_id)
2: if handle = None then return TERMINATE
3: s0 ← handle.get_status()
4: if s0.status ≠ active or s0.error_count
   ≥  $\theta_e$  then
5:   return RESTART
6: sleep( $\tau$ ); s1 ← handle.get_status()
7: if s1.uptime ≤ s0.uptime then return
   RESTART
8: return HEALTHY

enforce_verdict(agent_id, tier):
9: match tier:
10:   RESTART: handle.stop();
    handle.start()
11:   REVOKE: handle.stop();
    registry.revoke(agent_id)
12:   TERMINATE: ray.kill(handle);
    registry.unregister(agent_id)

```

7.8 Status Note

Hunter’s investigation method remains an unimplemented stub (four bare TODOs: query control-plane history, build an evidence chain, compute confidence, submit to Jury) at the time of writing — the one swarm role, by design, intended to be a fine-tuned agentic LLM tool-call loop rather than a rule or a numeric RL policy, and a substantially larger, separate undertaking not attempted in this development cycle.

8 kb-checker: Stateless Integrity Watchdog

kb-checker is a Rust subsystem deliberately kept stateless and network-free beyond its own local diagnostic socket, following a KISS design principle intended to minimize its own attack surface as the component responsible for detecting compromise of everything else.

8.1 Active Auditing

kb-checker performs three classes of active audit rather than passive log-watching. **Bytecode integrity**: it retrieves loaded BPF program metadata from the kernel, extracts the JIT-compiled instruction footprint via `bpf_obj_get_info_by_fd`, and computes a SHA-256 hash to verify it against an expected signature. **Map self-healing**: it traverses the live kernel containment map via `libbpf-sys`, cross-references its keys against the control plane’s database, and heals discrepancies with `bpf_map_update_elem`. **Liveness**: it connects to the control plane’s telemetry stream, forks a disposable test process (`/bin/true`), and verifies that the corresponding `sched_process_exec` event is observed within a bounded (3-second) window, catching a fully wedged sensor that is

still technically running but no longer actually emitting telemetry. Execution-latency audits additionally read JIT instruction run-count and cumulative run-time counters to flag hooks whose average execution time exceeds a 500ns threshold.

8.2 Boot-Sequence Gating

kb-checker gates the swarm’s ability to start at all. Per the project’s boot-sequence specification, `systemd` is configured such that if kb-checker’s audits fail, the Ray-based agent service (`kbd-agent.service`) is blocked from starting, effectively quarantining a host from executing swarm workloads until the watchdog is satisfied that the layers beneath it are intact.

8.3 Fallback Containment Hierarchy

For cases where the primary daemons cannot be cleanly unloaded via normal `systemd` control, kb-checker implements a three-layer hard fallback: force-killing the sensor’s userspace process group (`SIGKILL`), detaching dangling BPF hooks directly via `bpf tool` if the process-level kill is insufficient, and, as a last resort, an `iptables`-based network quarantine that drops all traffic except loopback, preserving local console access for an operator to intervene manually. §8.5 extends this exact chain with a fifth, hardware-anchored stage.

8.4 Status Note

kb-checker’s own test suite (`cargo test`) passes (one gRPC health-check integration test at the time of writing); no open defects were found within the time available for the audit in §11. kb-checker is also now included in CPM’s protected floor (§5), closing a gap where KB’s own containment pipeline could in principle be turned against its own watchdog.

8.5 Extended Feature: NOMAD Hardware Trust Module

kb-checker’s active auditing already anticipates that kb-core telemetry cannot be blindly trusted, but kb-checker itself remains a userspace process on the same physical machine, under the same kernel and scheduler as everything it watches — the same class of problem CPM solves for containment authorization (§5), restated at the hardware/trust-boundary layer. **The Node Out-of-Band Module for Attestation & Defense (NOMAD)** is an extended hardware feature addressing this directly: a physically independent appliance, split across an SBC (attestation/ policy), a dedicated STM32 watchdog/relay controller (continues firing even if the SBC’s Linux instance crashes), an ESP32 management processor, and a TPM 2.0 root of trust, attesting to and able to fence the host without depending on that host’s kernel being honest.

Unlike the rest of this paper’s subject matter, NOMAD is developed in a separate, parallel repository

(<https://github.com/pardhuvarmax/the-nomad>) by a dedicated hardware sub-team, not in this codebase, and is not a central concern of this paper — it is documented here only briefly, as one extended feature of the overall KB threat model. At the time of writing that repository shows real, active hardware bring-up (STM32 relay-control firmware and SPI display drivers, ESP32 firmware, and PCB manufacturing artifacts), with the SBC-side attestation software and the inter-processor wire protocol not yet started — an early-stage bench prototype under active parallel development, not a paper proposal with no code. Design highlights not yet reflected in running firmware: two physically separate communication planes (an Out-of-Band Trust Plane carrying attestation/heartbeat, and a Fabric Management Plane for fleet orchestration, which never share transport); hardware fencing positioned as a fifth, last-resort stage below kb-checker’s existing software recovery chain (§8); and a rootkit-resistance mechanism sealing kb-checker’s signing key to the host’s own TPM-measured boot state via TPM2_PolicyPCR [Trusted Computing Group, 2019, Sailer et al., 2004] so that a kernel module load invalidates the seal until reboot — addressing the confused-deputy scenario [Hardy, 1988] where kb-checker’s code stays intact while a compromised kernel forges what it reports. Full architecture, threat-model reasoning, and bill-of-materials costing live in that repository’s own documentation, not reproduced here.

9 Boot Sequence and Daemon Lifecycle

The five subsystems above do not start independently — boot order is a real security property here, not an operational convenience: an attacker who can start the agent swarm before the integrity watchdog has verified anything gets a window to act unsupervised. KB enforces boot order with ordinary systemd unit dependencies (After=/Requires=), not a custom supervisor, so the gate is exactly as trustworthy as systemd itself.

9.1 Three Phases

Phase 1 — core control plane. systemd starts kbd.service. kbd creates /run/kb/ and binds kbd.sock, kbct.sock, and kba.sock (Table 1), loads and attaches kbd_sensor’s eBPF programs, and sends an sd_notify readiness signal only once its sockets are actually listening — Type=notify in the unit file, not a fixed startup delay, so downstream services never race an kbd that looks started but isn’t yet accepting connections.

Phase 2 — safety layer activation. kb-checker.service declares After=kbd.service and Requires=kbd.service — it cannot start at all until kbd has reported ready. kb-checker acquires an exclusive POSIX flock on /run/kb/kb-checker.pid (a second instance started manually while the service is active fails this lock and aborts immediately, never contending with the real watchdog), binds kbc.sock,

and runs four *blocking* checks before declaring itself operational: a JIT bytecode SHA-256 hash comparison, a containment-map integrity diff against kbd’s own database, a liveness probe (inject a test process, require its exec event to stream back within 3s), and a hook-latency check (average execution time under 500ns).

Phase 3 — workload activation. kbagents.service (the Ray swarm node agent) declares Requires=kb-checker.service — the same mechanism, one level up. If Phase 2’s checks pass, systemd releases this dependency and the swarm registers itself over kba.sock; if kb-checker fails to start *at all*, systemd’s own dependency resolution blocks kbagents.service from ever starting — no custom gating code in KB decides this, the same guarantee systemd gives any unit with a Requires= on a unit that failed.

9.2 Fail-Safe Paths

Three failure modes are handled explicitly, not left to silent retries. An **eBPF hook loading failure** (no LSM support, or kbd_sensor fails to attach) means kbd never sends its readiness signal, so kb-checker — gated on that signal — never starts, and neither does the swarm. A **UDS socket loss** (e.g. kbd restarting mid-run, briefly deleting kba.sock) is tolerated for up to 15s of retries before kb-checker treats it as a containment-worthy event and forces an emergency bytecode reload. **Graceful termination** (systemctl stop kb-checker) is a real code path, not just process death: kb-checker catches SIGTERM, stops its audit loops, closes its gRPC server, and removes both kbc.sock and its own PID-lock file before exiting, so a deliberate stop never leaves stale state for the next start to trip over.

9.3 Chronological Timeline

Table 7 gives the same three phases as a single concrete timeline, with the approximate wall-clock offsets from power-on the boot-sequence specification records for a representative boot — the phase boundaries in Figure 3 are the After=/Requires= dependencies; this is how long each side of them actually takes in practice.

Table 7: Chronological boot sequence, power-on to full operational state.

Offset	Event
T+0.00s	Power-on: UEFI/GRUB verify and measure the boot chain; initramfs and root filesystem mount; systemd (PID 1) takes control
T+4.50s	Phase 1: kbd.service binds sockets, loads eBPF, sends sd_notify ready
T+6.00s	Phase 2: kb-checker.service spawns (After=kbd), locks its PID file, runs blocking audits
T+7.50s	Decision: pass → loops; fail → Containment Protocol
T+8.50s	Phase 3: kbagents starts if Phase 2 passed (else blocked) and registers over kba.sock
T+10.00s	Full operational state: all subsystems live

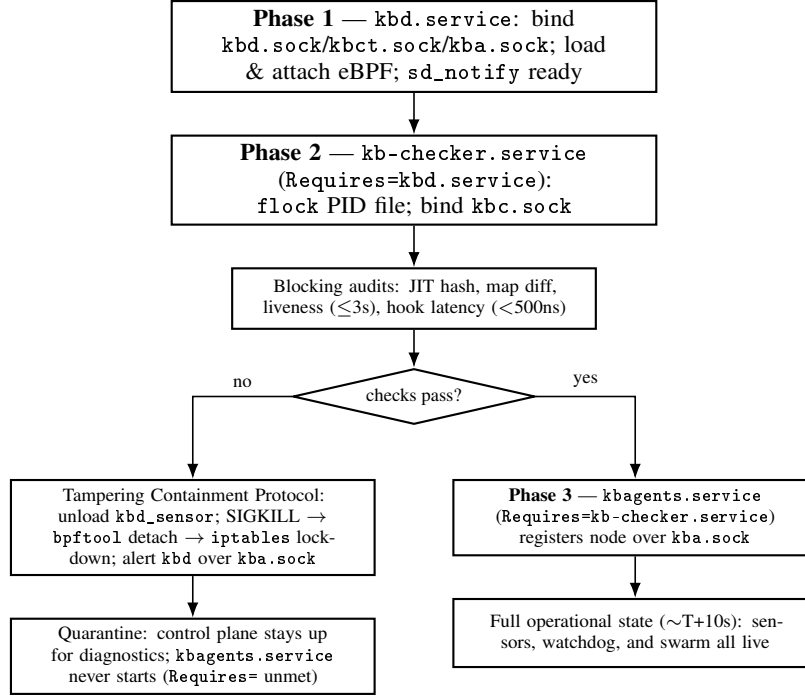


Figure 3: Boot sequence and daemon-lifecycle gating (§9). Every arrow crossing a phase boundary corresponds to a real systemd After=/Requires= dependency, not application-level polling — a failed or absent kb-checker structurally prevents kbagents.service from starting, the same way systemd would block any unit depending on a failed one.

10 kb-op: Operator Interfaces

kb-op provides four operator-facing surfaces over KB’s gRPC and HTTP APIs.

10.1 kbctl

kbctl is a Go CLI dialing kbd’s gRPC service directly over its Unix domain socket. It supports process isolation and restore (process isolate, with an explicit CGROUP|SECCOMP|NAMESPACE|TERMINATE level flag), zone classification overrides, audit-chain verification and JSON export (audit verify, audit export), policy hot-reload (policy reload), system statistics, and CWP workload key generation and signing (workload keygen, workload sign, §6.4). An earlier revision of this paper’s audit reported audit verify as calling `os.Exit(1)` directly on a detected chain break, bypassing its own deferred cleanup; current source returns the failure through Cobra’s normal RunE error path instead, letting every defer run — this claim did not hold up against current code and is corrected here rather than carried forward.

10.2 kb-tui

kb-tui is a Rust/ratatui terminal console. An earlier revision of this paper described it as reachable over a custom, hand-rolled Go SSH server hosted in-process by kbd (internal/ssh/), with a security migration to

real OpenSSH recorded only as a considered, not-yet-implemented future decision. That migration is, in fact, already complete: `internal/ssh/` has been deleted from kb-control-plane in full, and port 2222 is now served by a second, independent instance of the system’s own sshd (`sshd@kb-operator.service`), configured with ForceCommand to `exec kb-tui` directly after real OpenSSH authentication — kbd retains no host keys, `authorized_keys` parsing, or PTY code of its own. This resolves both defects an earlier revision of this audit attributed to the retired custom server (no login-attempt throttling; a dev-mode host key path that could silently relocate key material) by removing the component they applied to, not by patching it — real sshd’s own decades-old hardening (rate limiting via PAM/fail2ban conventions, fixed host-key handling) now governs this listener entirely. kbctl’s hidden `ssh` subcommand is the audit-callback glue this design still needs: it is invoked by the ForceCommand wrapper script (not run interactively) to record session metadata (principal, remote address, identity) against the audit log.

10.3 kb-dashboard

kb-dashboard is a TypeScript web frontend against kbd’s HTTP API. An earlier revision of this paper reported its isolate/restore controls as the client side of an unauthenticated, wildcard-CORS API with a hardcoded `localhost:8080` host. Current source does not match that description: the

API base resolves from `window.location.hostname` (overridable via `VITE_KB_API_BASE` for a non-local deployment) rather than a hardcoded host, and isolate/restore requests carry a bearer token from `VITE_KB_API_TOKEN` matching the server’s `KB_HTTP_API_TOKEN` (§6) — the dashboard’s own source comments this fix as tied directly to that server-side change. This is corrected here rather than repeated.

10.4 kb-mcp

kb-mcp exposes KB’s telemetry and query surface as a Model Context Protocol server, intended to let LLM-based tooling query process state and zone classifications through the same gRPC API `kbctl` and `kb-tui` use. This subsystem was not included in the source-level audit reported in §11.

11 Security Audit

Independent of any single subsystem’s feature work, we conducted a source-level audit across `kb-core`, `kb-control-plane`, `kb-aads`, and `kb-op` (`kb-checker` was included in scope but no defects were found within the time available, and `kb-mcp` specifically was not covered). An earlier revision of this audit reported fifteen open defects, five of them summarized in a severity table; re-verifying every one of those claims directly against current source for this revision — rather than continuing to carry them forward — found that the great majority no longer hold: they were already fixed in team history that simply predates when this paper’s audit text was originally drafted, and were never re-checked against current code before being repeated. This is, itself, worth stating plainly as a methodological finding: a security audit’s claims decay the moment the underlying code moves, and re-verifying against current source before every revision is not optional. Specifically confirmed already fixed, and corrected in this revision rather than left stale: the unauthenticated wildcard-CORS HTTP API (bearer-token auth plus an origin allowlist now gate it, §6), the telemetry-update containment-reset bug (`UpsertProcessState` explicitly preserves containment across updates, §6), the LSM level-3+ fail-open ordering (§5), the custom SSH server’s missing throttling and dev-mode host-key fallback (the server itself was deleted and replaced by `real sshd`, §10), the dashboard’s hardcoded API host (§10), absent input validation/rate limiting on containment RPCs (existence checks and per-agent throttling are tested, §6), `kbctl`’s claimed `os.Exit(1)` cleanup bypass (not present in current source, §10), and Jury’s three previously-reported consensus defects (§7.2). One claim was re-verified and confirmed genuinely still open: the audit-chain verifier’s missing `rows.Err()` check (§6.2), which is fixed in this revision with a dedicated regression test. At the time of writing we are not aware of any other confirmed-open defect in the four audited subsystems beyond what is discussed alongside each subsystem above; this reflects the scope and time

available for this revision’s re-verification pass, not a claim of exhaustive proof.

Table 8: Status of every defect this paper has, at some revision, reported as open. “Predates audit” means current source already fixed it before this paper’s audit text was first drafted.

ID	Status	Summary
BUG-001	Fixed (predates audit)	HTTP API had no auth, wildcard CORS (§6)
BUG-002	Fixed (predates audit)	Telemetry reset active containment to NONE (§6)
BUG-004	Fixed (predates audit)	LSM hook could fail open on path-resolution failure (§5)
BUG-005	Fixed this revision	Audit verifier never checked <code>rows.Err()</code> (§6.2)
BUG-006	Moot (component removed)	Custom SSH throttling gap; server since deleted (§10)

12 Current Project Status

Table 9 summarizes what is complete, partial, or outstanding across all five subsystems this paper covers at the time of writing. NOMAD (§8.5), developed in a separate repository, is excluded from this table.

13 Limitations and Future Work

Several gaps are explicitly acknowledged rather than papered over. KB’s own attack-lab data pipeline remains unbuilt, so Containment and Jury’s shared training data (§7.3, §7.4) is a public-dataset stand-in with no coverage for the `memory_exploit` scenario, and no mapping yet exists from `kb-core`’s real eBPF event-type vocabulary to the category codes that policy was trained on. Hunter requires both a fine-tuned agentic LLM tool-call loop and its own training data, neither of which exists yet. Healer’s decision remains rule-based, not learned, because the restore-outcome ground truth its reward would require does not exist in this codebase. The JJE courthouse’s severity classifier (Equation (10)) only ever reaches `restart` automatically; `revoke` and `terminate` require signal producers (peer-baseline drift detection, authorization-boundary violation detection) that have not been built, and its own quorum-gating design is not implemented for the reason given in §7.7. `kb-mcp` was not covered by this audit cycle. NOMAD (§8.5) is out of this paper’s own scope, tracked and developed independently; its parallel repository’s own documentation flags PCR-bank selection and re-sealing ownership, the SBC/STM32/ESP32 internal protocol, and the FMS wire protocol as unresolved design questions there, separate from anything this paper is responsible for. On the security-audit front specifically: this revision’s re-verification pass found that most previously-reported open findings were already fixed in code that predates when this paper’s audit text was first drafted, which is itself a limitation worth stating plainly — an audit’s claims are only

Table 9: Per-subsystem implementation status.

Subsystem	Status
kb-core (§5)	Complete for the enforcement primitives documented here, each independently live-verified, including a new userspace-only network-flow wire message for exfiltration detection. Both LSM fail-open-ordering and PID-keyed-map-exhaustion claims from an earlier audit revision are confirmed already fixed on re-verification.
kb-control-plane (§6)	Complete for storage, scoring ingestion, gRPC/HTTP API, audit-chain logging, policy hot-reload, signed CWP workload policy, severity escalation, and exfiltration/ beaconing detection. 128 tests passing. HTTP API auth/CORS and the containment-reset bug were already fixed in prior history; the audit-chain <code>rows.Err()</code> gap is fixed in this revision.
kb-aads (§7)	Actor scaffolding functional. Containment and Jury roles carry trained, live-verified RL policies. Patroller and Healer carry real rule-based logic. Containment militia, signal relays, and <code>SwarmRegistry</code> -backed JJE courthouse oversight are newly implemented. Hunter remains a non-functional stub, by design a separate, larger undertaking. 68 tests passing.
kb-checker (§8)	Complete for its stated scope (stateless integrity auditing, boot-sequence gating, fallback containment), now included in CPM’s protected floor. No defects found in this audit within the time available.
kb-op (§10)	<code>kbctl</code> , <code>kb-tui</code> , and <code>kb-dashboard</code> functional against the current gRPC/HTTP surface; <code>kbctl</code> gained CWP workload key generation/signing. <code>kb-tui</code> ’s SSH transport migrated to a real, independent <code>sshd</code> instance, superseding and removing the custom Go SSH server this paper’s earlier audit revision evaluated. <code>kb-mcp</code> functional per its own scope but not included in this audit’s coverage.

as current as the last time they were checked against real source, and this paper’s own earlier revision went uncorrected for some time. The one defect re-verification confirmed genuinely open (§6.2’s `rows.Err()` gap) is fixed in this revision. We are not aware of a remaining critical, unauthenticated enforcement surface in current source, but this reflects the scope and time available for this pass, not a claim of exhaustive security review; a fresh, independent audit before any production deployment remains warranted regardless.

14 Conclusion

We documented Kernel Borderlands subsystem by subsystem: kernel-verified enforcement primitives and authorization gates in `kb-core`; a two-tier storage engine, hash-

chained audit log, hot-reloadable policy engine, signed workload policy, and statistical exfiltration detection in `kb-control-plane`; an actor-based swarm in `kb-aads` now carrying two trained and live-verified reinforcement-learning decision roles (Containment and Jury), real rule-based Patroller and Healer logic, a mechanically-sequenced containment militia, dedicated signal-relay actors, and a registry-backed courthouse oversight mechanism for the swarm itself; a stateless boot-gating watchdog in `kb-checker`, now extended with a briefly-noted, separately and actively developed hardware trust module, NOMAD, addressing the one failure mode no same-host software component can structurally cover (total kernel compromise), though not itself a subject this paper documents in depth; and four operator interfaces in `kb-op`. We gave a single, consolidated mathematical formulation covering every learned and statistically-driven decision procedure in the system, and subjected the four implemented, non-hardware subsystems to the same source-level security scrutiny this paper’s audit reports throughout. Re-verifying that audit’s own prior claims against current source for this revision — rather than repeating them — found the great majority already resolved, fixed one genuine remaining defect (an audit-chain integrity check silently masking a class of I/O error) with a dedicated regression test, and corrected several since-superseded claims (a migrated SSH transport, an already-authenticated HTTP API) that an earlier revision had continued to describe as open or as future work. A fresh, independent audit before any production deployment remains warranted regardless of this finding.

References

- Kernel Borderlands Team. Kernel borderlands: Kernel-level runtime defense framework. <https://github.com/pardhuvarmax/kernel-borderlands>, 2026.
- Brendan Gregg. BPF performance tools. In *Addison-Wesley Professional Computing Series*, 2019.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. RLlib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning*, pages 3053–3062. PMLR, 2018.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.
- Claude E Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

- Ewan S Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, 2010.
- Hung-Jen Liao, Chun-Hung Richard Lin, Ying-Chih Lin, and Kuang-Yuan Tung. Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1):16–24, 2013.
- Gideon Creech and Jiankun Hu. Generation of a new ids test dataset: Time to retire the kdd collection. *2013 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 4487–4492, 2013.
- Trusted Computing Group. Trusted Platform Module Library, family “2.0”, level 00, revision 1.59. Technical report, Trusted Computing Group, 2019.
- Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert Van Doorn. Design and implementation of a TCG-based integrity measurement architecture. In *13th USENIX Security Symposium*, pages 223–238, 2004.
- Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.